CS7643: Deep Learning
Assignment 4

March 15, 2025

- This assignment is due on the date/time posted on canvas. We will have a 48-hour grace period for this assignment. However, no questions regarding the assignment are answered during the grace period in any form.
- Discussion is encouraged, but each student must write his/her own answers and explicitly mention any collaborators.
- Each student is expected to respect and follow the GT Honor Code. **We will apply anti-cheating software to check for plagiarism**. Anyone who is flagged by the software will automatically receive 0 for the homework and be reported to OSI.
- Please **do not change the filenames and function definitions** in the skeleton code provided, as this will cause the test scripts to fail and you will receive no points in those failed tests. You may also **NOT** change the import modules in each file or import additional modules.
- It is your responsibility to make sure that all code and other deliverables are in the correct format and that your submission compiles and runs. We will not manually check your code (this is not feasible given the class size). Thus, **non-runnable code in our test environment will directly lead to a score of 0**. Also, your entire programming parts will **NOT** be graded and given 0 score if your code prints out anything that is not asked in each question.

Code: Generative models

The objective of this assignment is to implement and experiment with various generative models. Large sections of the code base including each of the trainers are already implemented with certain TODO blocks where further implementation is required. The focus is on using implemented code for experiments on TWO datasets: MNIST and Fashion-MNIST. MNIST should be familiar already. Fashion-MNIST is a similar 28×28 image dataset (10 categories) across various clothing categories.

Setup

We recommend `python (3.11), torch (2.6.x) and corresponding torchvision`. Most versions should work as long as `torch > 2.0` which requires `python > 3.7` however they are untested. The following libraries are required for this assignment `pytorch, torchvision, pyyaml, matplotlib`.

Download data by running `python download_data.py` in the root directory of the assignment. This is important because the download will occur relative to where the file ran from. Both MNIST and FashionMNIST will be downloaded.

Review the config file `configs/config.yaml` provided and get comfortable with the parameters as it drives the trainer `train.py`. Note that not all parameters are used for every model, only relevant parameters will be used by each method. These are often commented directly in the config but is nonetheless straightforward. Each model type has its individual trainer (eg. `diffusion_trainer.py`) which is invoked based on the config.

To run the trainer: `python train.py --config_file $PATH_TO_CONFIG_FILE`.

When a model is trained, a folder outputs will be generated with the particular model name, eg. `outputs/vae/`.

Note that we also provide a `generate_images.py` file that samples a respective model and generates samples. It is used similar to the trainer:

eg. `python generate_images.py --config $PATH_TO_CONFIG_FILE`

1 Preliminaries

Recall that discriminative models aim to model $p(y|x)$, that is the conditional distribution of the probability of the label given the input. Generative models on the other hand aim to model the distribution over the input space $p(x)$. ie. we do not require labels and are trying to model the data itself. This is not the only use of the term "generative model" but the one we use here.

How do we do this? One way is we can parameterize models $p_\theta(x)$ and use maximum likelihood to optimize the parameters on an unlabeled dataset. We'll start with such a paradigm and choose to parameterize our model using neural networks.

The following readings may be useful for additional context around the generative models we explore:

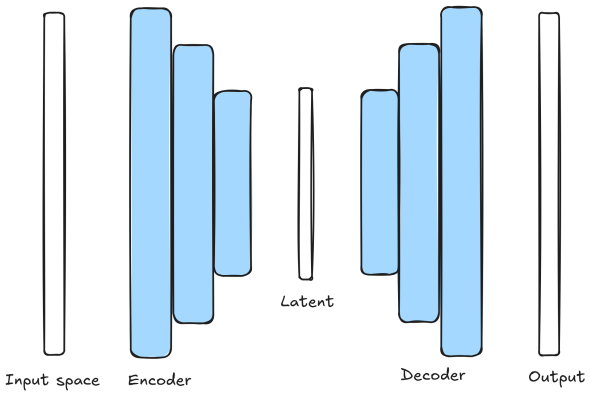


Figure 1: traditional autoencoder. Typically is optimized using a reconstruction objective between input and output.

- Tutorial on VAEs
- Tutorial on GANs
- Denoising Diffusion Probabilistic Models

2 Variational Autoencoder

A traditional autoencoder aims to learn a set of parameters that efficiently reconstructs the data. First inputs are encoded into a code/latent representation and then decoded to the output space, reference fig. 1. A reconstruction objective between the input and output typically drives this process. Typically the dimensionality of the latent code is smaller than the input, motivating the optimization to reduce the dimensionality of data. This can be useful as this compact representation of the data may transfer well to downstream tasks.

Variational Autoencoders (VAE) are probabilistic view on the autoencoder (reference fig. 2). We want the compressed representation from an autoencoder to truly represent the latent variables of the distribution. That is that the latent space z is a smaller set of factors of variation in our data. eg. for MNIST we have a notion of scale, rotation, number etc. We view these latent variables as being from some distribution themselves, so if we were to sample from the distributions governing the latents they would map to some meaningful data back in the input space. While there is no constraint on *which* distribution we pick as a prior for the latents gaussian is a common one.

Supposing we sample our latents $z \sim \mathcal{N}(\mu = 0, \sigma^2 = 1)$, a challenge we have is that we cannot backpropagate through this sampling process fig. 2. To overcome this we use the *reparameterization trick*, we reparameterize z as a deterministic function of μ and σ^2 and a random variable ϵ. z is decoded by the decoder back to the output space and the optimization is driven by a reconstruction objective between input and output, similar to a traditional autoencoder, yet we still need to regularize to ensure the distribution we are learning stays close to our prior.

More concretely, given our prior that the latent space forms a gaussian distribution, the encoder $q_\phi(z|x)$ approximates the posterior distribution of the latent variables given

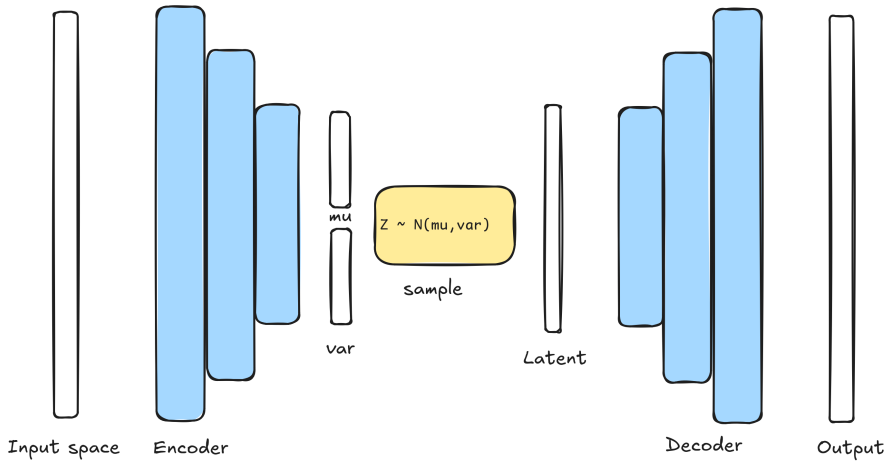


Figure 2: variational autoencoder. Notice that this sampling procedure complicates backpropagation. To alleviate this, we use reparameterization trick.

the input data. The decoder $p_\theta(x|z)$ approximates the likelihood of reconstructing the input data given the latent z. This allows us to generate samples from the distribution by sampling latents z from our prior and letting the decoder map them to the input space.

For VAE you need to implement the following:

1. BasicEncoder module in `models/VAE.py`
2. VAE module in `models/VAE.py`
3. VAE loss in `losses/vae_loss.py`
4. VAE training loop in `trainer_vae.py`

2.1 VAE encoder

We are given the implementation of the decoder from `models/decoder.py`. You will have to implement the encoder.

The encoder architecture consists of a feed-forward neural network with the following structure in figure 3. It takes the input vector and encodes it into two vectors, μ and $\log \sigma^2$ which represent the parameters of the distribution over z.

Note: The choice to output $\log \sigma^2$ rather than σ^2 directly involves important numerical considerations during optimization. Consider the properties of log and how it relates to gradient flow and numerical stability.

Complete the TODO's in the BasicEncoder module from `models/VAE.py`.

2.2 Variational Autoencoder

With the encoder and decoder defined we can use them in our VAE. Complete the VAE module

1. Instantiate the decoder and encoder in the VAE init.

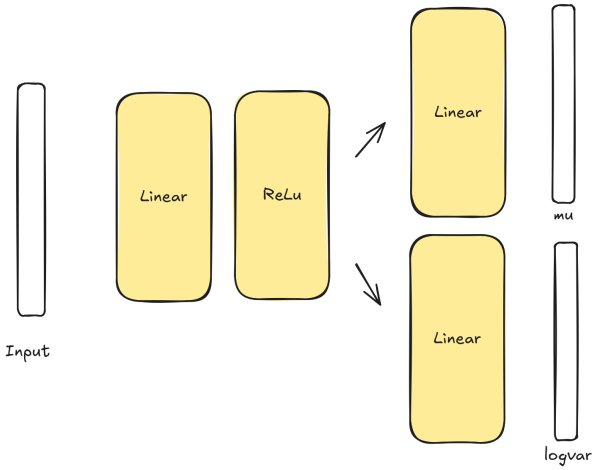


Figure 3: VAE encoder diagram.

2. Implement the VAE encode forward. this is simply using the encoder you defined to obtain μ and $\log \sigma^2$
3. Implement the reparameterization forward.
4. Complete the forward function for the VAE using the defined functions.

Since we cannot pass gradients through a sampling procedure we use the reparameterization trick. This trick transforms the latents μ and $\log \sigma^2$ into a sample z in a manner we can pass gradients.

The key idea is to express $z \sim \mathcal{N}(\mu, \sigma^2)$ as a deterministic function of μ, σ, and a random variable $\epsilon \sim \mathcal{N}(0, I)$. Hence, $z = \mu + \epsilon\sigma$, this transformation allows gradients to flow during backprop since ϵ is independent of the learnable parameters.

2.3 VAE loss

The VAE loss has 2 components. The first is a reconstruction loss between the input x and output of our encoder.

$$\mathcal{L}_{recon} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] \tag{1}$$

Which is typically as $\mathcal{L}1$ or $\mathcal{L}2$. NOTE: since pytorch aggregates the loss over both batch and spatial dimensions, we choose to add the spatial scale ($28 \times 28 = 784$) back in. This is required to pass the loss unit test.

In addition we will use a KL-divergence term to regularize the latent space to be structured close to our prior, that is that the output of our encoder $q(z|x)$, μ and $\log \sigma^2$ form a gaussian. The KL term is as follows:

$$\mathcal{L}_{KL} = D_{KL}(q_\phi(z|x)||p(z)) \tag{2}$$

Under a gaussian prior, 2 simplifies to:

$$\mathcal{L}_{KL} = -\frac{1}{2}\sum_{i=1}^{d}(1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2) \tag{3}$$

To compute the total loss, we take the sum of the reconstruction and KL loss terms. However we need to balance the relative scale of the two terms. This is done via a β parameter:

$$\mathcal{L}_{total} = \mathcal{L}_{recon} + \beta\mathcal{L}_{KL} \tag{4}$$

complete the VAE loss in `losses/vae_loss.py`

2.4 VAE training loop

Similar to any other network, we need to write a training loop to optimize the parameters of the VAE. Complete VAE training loop in `trainer_vae.py`

To train the model. Update the config in `configs/config_vae.yaml` with the desired hyperparameters and run using `python train.py --config configs/config_vae.yaml`

2.5 VAE - Experiments

During training reconstruction images will be saved. In addition fixed latents z_s are sampled mapped to the data space and saved so we can observe qualitatively how well our decoder is able to generate samples.

2.5.1 β and reconstruction loss experiments

Using the two datasets provided, tune the β parameter to explore its effect. Then compare the results of $\mathcal{L}1$ vs $\mathcal{L}2$ reconstruction loss. Note: the change between reconstruction losses is simply a change in the config file,

2.5.2 MNIST few-shot KNN

change the config to MNIST and tune your parameters to pass the KNN tests. Here we are testing how "good" a representation the encoder has learnt by training a K Nearest Neighbors (KNN) classifer on 10 samples per class (100 total samples) from the MNIST dataset. In this low data regime it is natural to expect a performance drop, although perhaps the unsupervised learning from the VAE has let the encoder capture a good representation of the data.

As we optimize for our few-shot setting what happens to the quality of generated samples?

2.5.3 Fashion-MNIST FID

Change your config to fashionMNIST and tune your parameters to achieve the lowest FID score as possible.

After training your model, generate test images using

`python generate_images.py --config_file configs/config_vae.yaml`

and then measure the FID between your generated images and the test images:

`python -m pytorch_fid output/vae/images tests/assets/fid-stats-fashion.npz --device <your-device>`

Record your observations of the generated samples as you tune parameters. Frechet Inception Distance is a measure of the quality of generated images, how does this metric compare to your qualitative assessment of the generated samples?

3 Generative Adversarial Networks (GANs)

VAEs explicitly define and learn the data distribution $p(x)$ through likelihood estimation which has certain drawbacks you may have observed during earlier experiments. GANs trade likelihood estimation for adversarial training and implicitly learn to sample from $p(x)$. This is done by pitting two networks in a zero-sum game, the generator G which generates samples against a discriminator D which outputs the probability the generates samples are real. Specifically the generator $G_\theta(z)$ seeks to transform a sample latent z into a sample from the data distribution $p(x)$ such that the discriminator $D_\phi(G_\theta(z))$ is fooled.

This adversarial objective is defined as follows:

$$\min_G \max_D \mathbb{E}_{x\sim p(x)}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))] \tag{5}$$

The GAN training loop consists of two main phases that alternate during training:

Discriminator Training:

- Process real data batch and compute discriminator's performance on real samples
- Generate synthetic data using current generator state and compute discriminator's performance on fake samples
- Update discriminator parameters based on its ability to distinguish real from fake

Generator Training:

- Generate new synthetic data samples
- Update generator parameters based on discriminator's assessment

Overall you need to implement the following:

1. BasicDiscriminator module in `models/GAN.py`
2. compute_loss_real function in `trainer_gan.py`
3. compute_loss_fake function in `trainer_gan.py`
4. compute_loss_gen function in `trainer_gan.py`
5. GAN training loop in `trainer_gan.py`.

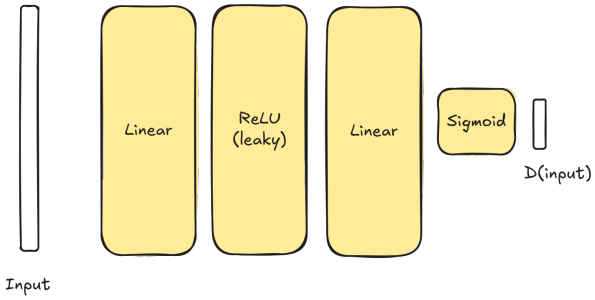


Figure 4: GAN discriminator diagram.

3.1 GAN discriminator

You will implement the discriminator whereas the GAN generator will be provided. We are using the same provided generator as in VAE except the ReLU can optionally be replaced with LeakyReLU (from the config).

The discriminator follows a basic feedforward network structure that outputs a probability between 0 and 1 (probability of the input data being fake), see figure 4.

3.2 GAN losses

Implement the GAN losses as follows:

1. compute_loss_real function in `trainer_gan.py`
2. compute_loss_fake function in `trainer_gan.py`
3. compute_loss_gen function in `trainer_gan.py`

You'll notice instead of processing a batch of real and fake images together, we process them independently and will sum them in the training loop. This is a trick from GAN Hacks.

3.3 GAN training loop

Complete GAN training loop in `trainer_gan.py`.

After completing the aforementioned implementations train the GAN. Update the gan config in `configs/config_gan.yaml` with the desired hyperparameters and run using:

```
python train.py --config configs/config_gan.yaml
```

Every few epochs during training, latents z_s are sampled, passed through the generator and saved as images.

3.4 GAN - experiments

Tune the parameters across both both MNIST and Fashion-MNIST and achieve the lowest FID for the latter.

We used a VAE in a few shot scenario by using the encoder to embed our data, consider whether this can be done with GANs?

4 Diffusion Probabilistic Models

A diffusion probabilistic model learns to transform gaussian noise into a sample from the data distribution. This is done using a forward process to gradually add noise to a clean sample until it resembles gaussian noise. A reverse process learns to denoise starting from gaussian noise gradually into a sample from the distribution. We parameterize this reverse process with our network, whereas the forward process is fixed according to a noise schedule over T timesteps.

4.1 Simple "Diffusion"

First we will implement a simplified "trivial" version of diffusion or rather denoising. An implementation of a basic denoising network is provided and you will implement the denoising procedure. We first sample noise and add it to a fixed image, and our objective is to learn to predict noise such that we can denoise to recover the original signal.

The denoising training loop follows these key steps:

1. Given a clean image x, create a noisy version by adding noise ϵ:

$$\tilde{x} = x + \epsilon$$

2. Use the model to predict the noise component:

$$\hat{\epsilon} = f_\theta(\tilde{x})$$

3. Train the model to match the true noise:

$$\mathcal{L} = \|\hat{\epsilon} - \epsilon\|_2^2$$

For denoising at inference time:

1. Given a noisy image $\tilde{x}$, predict the noise component using the trained model:

$$\hat{\epsilon} = f_\theta(\tilde{x})$$

2. Remove the predicted noise to recover the denoised image:

$$\hat{x} = \tilde{x} - \hat{\epsilon}$$

You should implement the training step and reverse sampling step in `simple_diffusion.py`. To train the model run `simple_diffusion.py`. NOTE: no config necessary. During training images will be produced that show original image, noised image, noise that wadded, predicted noise, and denoised image.

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Figure 5: DDPM algorithm

5 Denoising Diffusion Probabilistic Models (DDPMs)

Now that we have a fundamental understanding of using a DNN to denoise images by predicting noise we can generalize this concept by learning to denoise over a schedule of noise that varies temporally. We follow the DDPM formulation as per fig. 5. Read `https://arxiv.org/pdf/2006.11239` for more context around DDPMs.

We write the forward gradual noising process as:

$$x_t \sim \mathcal{N}(\sqrt{1-\beta_t} \cdot x_{t-1}, \beta_t I) \tag{6}$$

and the reverse denoising process:

$$x_{t-1} \sim \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \tag{7}$$

The DDPM algorithm consists of two main procedures: training and sampling.

Training Loop:

- Sample data points and time steps.
- For each sampled pair, compute forward diffusion to obtain noisy versions using a noise schedule α_t
- Train the model ϵ_θ to predict the noise

Sampling Loop:

- Start from pure noise $x_T \sim \mathcal{N}(0, \mathbf{I})$
- Iteratively denoise using the learned model through a reverse process
- Return the final denoised sample x_0

We provide you a UNet implementation that will serve as the architecture for the diffusion model. We ask you to implement the forward and reverse diffusion processes.

Implement the TODO sections in *trainer_diffusion.py*

To train the model. Update the diffusion config in `configs/config_diffusion.yaml` with the desired hyperparameters and run using:

```
python train.py --config configs/config_diffusion.yaml
```

5.1 Noise Schedule

β_t is the variance schedule which is considered a set of constant hyperparameters. the upper and lower bound are set in the config, In the Diffusion Trainer `__init__` you will compute a linear set of β's between the lower and upper values of the schedule. In addition the α_t and $\bar{\alpha}_t$ vectors can be computed from β_t and cached for later use.

$$\alpha_t = 1 - \beta_t \tag{8}$$

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \tag{9}$$

Implement the noise schedule in `trainer_diffusion.py`

5.2 Forward diffusion

Given an input image x_0 and a timestep t, we want to compute the respective noised image for that timestep. Given equation 6, and noting a normal distribution can be written as $X = \mu + \sigma \cdot \epsilon$ where ϵ is some normally distributed noise, we can write the expanded form of the noising step as:

$$x_t = \sqrt{\bar{\alpha}} x_0 + \sqrt{1 - \bar{\alpha}} \epsilon \tag{10}$$

Implement the forward diffusion noising step in `trainer_diffusion.py`. Note that we need to return both the noised image and the raw noise that was added.

5.3 Diffusion training loop

Implement the training loop for diffusion. Reference training loop 5 and figure 5 (algorithm 1). Recall the basic components of a training loop in pytorch, this step should look very similar with some key differences, eg. inclusion of forward diffusion to obtain the noised image and noise added.

Implement the train TODO block in `trainer_diffusion.py`.

5.4 Sample Time Step

After training the network to predict the noise conditioned on a given timestep, we want to use this to perform the reverse diffusion process. First we will implement a single step of reverse diffusion, that is a single iteration of the loop shown in figure 5 (algorithm 2); given an image and the current timestep take a single step of reverse diffusion and compute the denoised image.

Implement the `sample_timestep` TODO block in `trainer_diffusion.py`.

5.5 Sample

After completing a single step of reverse diffusion, we can perform the full reverse diffusion process per figure 5 (algorithm 2).

Implement the `sample` TODO block in `trainer_diffusion.py`. HINT: You will need to iteratively call `sample_timestep`.

5.6 Diffusion - Experiments

As the diffusion model is trained. Forward diffusion and reverse diffusion visuals will be periodically saved. Tune the diffusion model across both datasets (MNIST and fashion-MNIST) and acheive the lowest FID for the latter dataset.

6 Experiments

Review the provided report template and follow the experiments shown.